

IPV – Instituto Politécnico de Viseu
ESTGV – Escola Superior de Tecnologia e Gestão de Viseu Departamento
de Informática



Relatório da unidade curricular Projeto Integrado

Licenciatura em Engenharia Informática

Realizado em
Projeto Integrado

Por

Claúdio Filho pv19866

Inês Silva pv25222

Pedro Rodrigues pv24512

Entidade: ESTGV

Docente: Artur Sousa

Luís Soares

Júlio Florentino

Viseu, 2025

Índice

Índice de Figuras.....	3
1.Introdução.....	5
2. Modelo De Dados.....	6
3. Plataforma web.....	7
3.1 Organização da estrutura do projeto	8
3.2 Login.....	9
3.3 Primeira Autenticação e Alteração da Palavra-passe.....	11
3.4 Escolha do perfil.....	14
3.5 Pedido de conta e validação do mesmo	15
3.5.1 Pedido de Conta – Interface e Lógica (Frontend)	16
3.5.2 Registo de Pedido (Backend)	17
3.5.3 Gestão do pedido - Interface e Lógica (Frontend)	19
3.5.1 Gestão do pedido (Backend)	22
3.5.2 Envio Automático de Email com Credenciais.....	24
3.6 SlideBar Administrador	27
3.7 Slidebar da área Formador e Formando.....	29
3.8 Gestão de cursos Administrador	30
3.8.1 Criação de novos cursos.....	30
4. Plataforma Mobile	34
5. Conclusão	34

Índice de Figuras

FIGURA 2-1-MODELO DE DADOS	7
FIGURA 3-1-ESTRUTURA MODULAR DO FRONTEND	8
FIGURA 3-2-ESTRUTURA MODULAR DO BACKEND	9
FIGURA 3-3- INTERFACE DO FRONTEND: PÁGINA DE LOGIN	9
FIGURA 3-4- CÓDIGO DA IMPLEMENTAÇÃO DO LOGIN (FRONTEND)	10
FIGURA 3-5 - CONTROLLER BACKEND DO LOGIN	11
FIGURA 3-6 - ALTERAÇÃO DE PALAVRA-PASSE NO PRIMEIRO LOGIN	12
FIGURA 3-7- CÓDIGO: ALTERAÇÃO DA PALAVRA-PASSE NO PRIMEIRO LOGIN (FRONTEND).....	12
FIGURA 3-8- CONTROLLER BACKEND: ALTERAÇÃO DA PALAVRA-PASSE NO PRIMEIRO LOGIN.....	13
FIGURA 3-9- INTERFACE DE SELEÇÃO DE PERFIL	14
FIGURA 3-10-CÓDIGO DA INTERFACE DE SELEÇÃO DE PERFIL (FRONTEND).....	15
FIGURA 3-11-CÓDIGO RESPONSÁVEL POR DEVOLVER OS PERFIS DISPONÍVEIS.....	15
FIGURA 3-12-INTERFACE DE PEDIDO DE CONTA (FRONTEND)	16
FIGURA 3-13-EXCERTO DE CÓDIGO DO COMPONENTE SOLICITARCONTA.JS	17
FIGURA 3-14 -EXCERTO DE CÓDIGO DO CONTROLLER SOLICITARCONTA	18
FIGURA 3-15-INTERFACE DE GESTÃO DE PEDIDOS NA LISTAGEM DE UTILIZADORES	20
FIGURA 3-16- FUNÇÃO PARA REJEIÇÃO DE PEDIDO	21
FIGURA 3-17-FUNÇÃO PARA ACEITAR PEDIDO DE CONTA	22
FIGURA 3-18-CONTROLLER PARA ATUALIZAÇÃO DO PEDIDO	23
FIGURA 3-19-CONTROLLER ACEITARPEDIDOCONTA	24
FIGURA 3-20-EXEMPLO DE ESTRUTURA NO .ENV	24
FIGURA 3-21-CONFIGURAÇÃO DO TRANSPORTADOR DE EMAIL	25
FIGURA 3-22-FUNÇÃO PARA ENVIO AUTOMÁTICO DE CREDENCIAIS POR EMAIL	25
FIGURA 3-23- INTEGRAÇÃO DO ENVIO DE EMAIL NO PROCESSO DE ACEITAÇÃO DE CONTA	26
FIGURA 3-24-SIDEBAR DO ADMINISTRADOR EM MODO EXPANDIDO	27
FIGURA 3-25-SIDEBAR DO ADMINISTRADOR EM MODO COLAPSADO	27
FIGURA 3-26-DEFINIÇÃO DOS ITENS DE MENU DA SIDEBAR DO ADMINISTRADOR COM ROTAS, ÍCONES E ETIQUETAS	28
FIGURA 3-27-SIDEBAR DO FORMANDO	29
FIGURA 3-28-SIDEBAR DO FORMADOR	29
FIGURA 3-29-INTERFACE DO FORMULÁRIO UTILIZADO PELO ADMINISTRADOR PARA A CRIAÇÃO DE NOVOS CURSOS	30
FIGURA 3-30-FUNÇÃO ASSÍNCRONA QUE ENVIA OS DADOS DO FORMULÁRIO PARA CRIAR UM CURSO	31
FIGURA 3-31-FUNÇÃO QUE REALIZA O ENVIO DO CONTEÚDO MULTIMÉDIA RELACIONADO AO CURSO PARA A API, GARANTINDO O ARMAZENAMENTO CORRETO DO FICHEIRO	31

FIGURA 3-32-CONTROLLER RESPONSÁVEL PELA VALIDAÇÃO E CRIAÇÃO DE UM NOVO CURSO NA BASE DE DADOS, DEFININDO O TIPO DE CURSO E O ESTADO INICIAL	33
FIGURA 3-33-CONTROLLER QUE PROCESSA O UPLOAD DE MÚLTIPLOS FICHEIROS PARA O ARMAZENAMENTO NO FIREBASE E REGISTA OS CONTEÚDOS NO SISTEMA, ASSOCIANDO-OS AO CURSO CORRESPONDENTE.	34

1.Introdução

O projeto *The SoftSkills* surge da necessidade crescente de promover a aprendizagem contínua e a partilha de conhecimento entre os colaboradores da Softinsa, especialmente num contexto de trabalho cada vez mais remoto e descentralizado. Com a dispersão geográfica das equipas e a redução do contacto presencial, tornou-se evidente a importância de uma plataforma digital que centralize o percurso formativo dos colaboradores e fomente a colaboração e o desenvolvimento das soft skills.

Este relatório apresenta o desenvolvimento de uma aplicação Web e Mobile com o objetivo de facilitar a inscrição em formações internas, o acesso a conteúdos educativos, e a partilha de conhecimentos entre colegas das mais diversas áreas. A plataforma visa não apenas apoiar a evolução profissional de cada colaborador, mas também criar um ambiente mais participativo, interativo e alinhado com a cultura organizacional da Softinsa.

A solução proposta oferece funcionalidades como a consulta e inscrição em cursos, gestão de conteúdos por formadores, avaliação formativa, fóruns de discussão temáticos e áreas de partilha por categoria de formação. Adicionalmente, será possível acompanhar todo o percurso formativo individual, incluindo certificados, notas, e número de horas completadas, contribuindo assim para uma gestão mais eficiente e transparente da formação interna.

O desenvolvimento do projeto envolveu a utilização de tecnologias modernas e boas práticas tanto no backend como no frontend. No backend, foi dada especial atenção à segurança, à gestão de permissões e à estruturação de dados. No frontend, procurou-se garantir uma experiência de utilizador fluida, responsiva e intuitiva, acessível a todos os perfis da organização — desde formandos a administradores.

Este relatório detalha todas as etapas do desenvolvimento da plataforma, desde a definição de requisitos até à implementação, destacando as decisões técnicas e funcionais que tornaram possível a criação de um sistema eficaz, colaborativo e adaptado às necessidades reais da empresa.

2. Modelo De Dados

As tabelas exibidas na Figura 2-1 fazem parte da base de dados da aplicação e cada uma é responsável pelo armazenamento da informação.

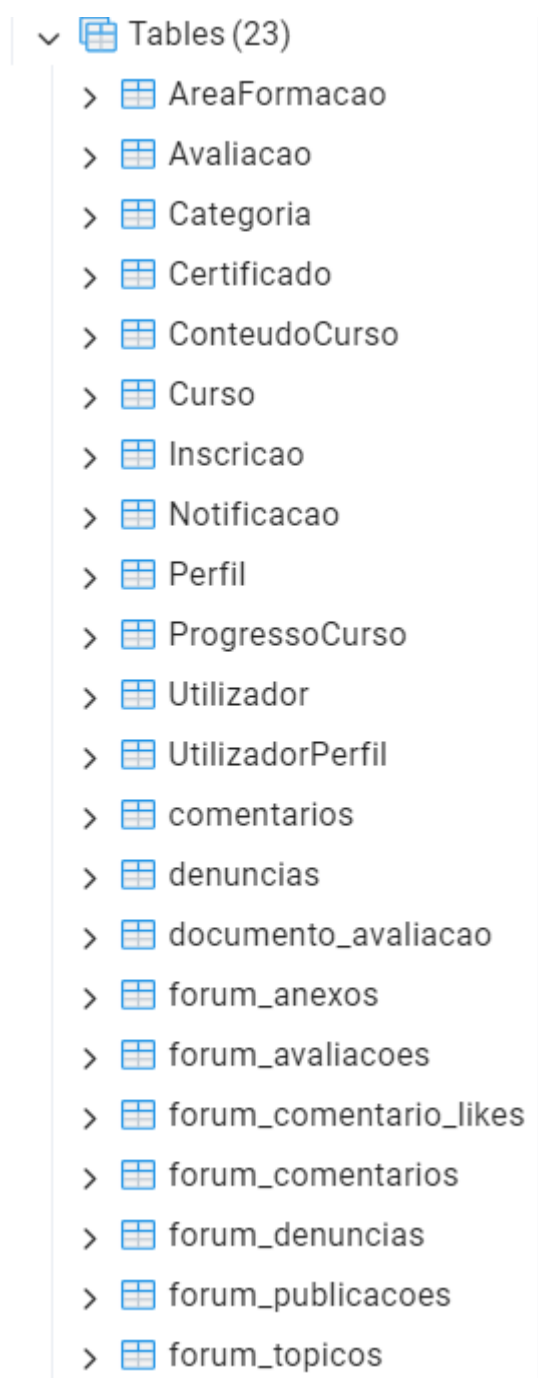


Figura 2-1-Modelo de dados

3. Plataforma web

A versão web do **The SoftSkills** oferece todas as funcionalidades da aplicação mobile, com o benefício adicional de um ambiente mais completo para formadores e administradores. Através da plataforma web, é possível gerir utilizadores e permissões, criar cursos e definir conteúdos, acompanhar a evolução dos formandos e organizar fóruns de partilha de conhecimento. Inclui também a disponibilização de ferramentas para avaliar formandos, gerar certificados e visualizar

o percurso formativo de cada colaborador, garantindo uma gestão eficaz e centralizada da formação interna.

3.1 Organização da estrutura do projeto

Esta camada da aplicação foi estruturada de forma modular, sendo organizada em 2 pastas:

- Frontend: Este é organizado em **três pastas principais**, conforme representado na **Erro! A origem da referência não foi encontrada.:**
 - ✓ **Componentes:** Esta pasta contém os vários componentes reutilizáveis utilizados em diferentes partes da aplicação, como, por exemplo, as *sidebars*
 - ✓ **CSS:** Esta pasta armazena os ficheiros de estilo responsáveis pela aparência da interface.
 - ✓ **Views:** A pasta “views” inclui os ficheiros que definem a estrutura e lógica das páginas da aplicação. Cada *view* representa uma página específica, implementada com recurso ao framework **React**. Estas páginas são compostas por componentes funcionais que gerem o estado local, lidam com eventos do utilizador e comunicam com a API backend.

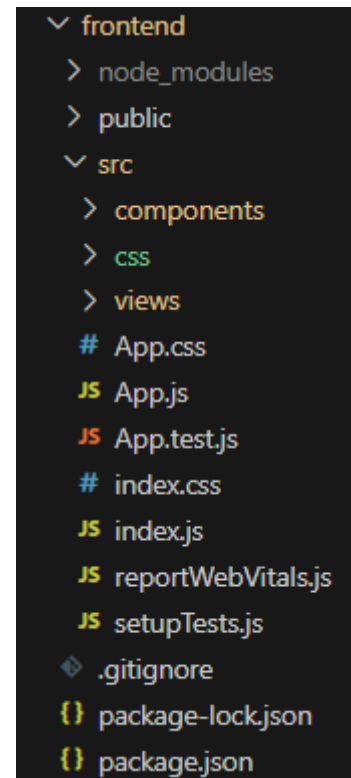


Figura 3-1-
Estrutura modular do
frontend

- Backend: Este é organizado em **três pastas principais**, conforme representado na Figura 3-2 :

- ✓ Config: Contém as configurações de acesso e utilização da base de dados alojada na plataforma Aiven.
- ✓ Controllers: Responsáveis pela lógica de acesso e manipulação de dados na base de dados.
- ✓ Firebase: Diretório destinado à integração com a plataforma Firebase, que é utilizada para o armazenamento de ficheiros da aplicação.
- ✓ Models: Define os modelos e estruturas das tabelas utilizadas na base de dados da aplicação.
- ✓ Routes: Responsável pela definição das rotas da aplicação, estabelecendo a comunicação entre o *frontend* e o *backend*.

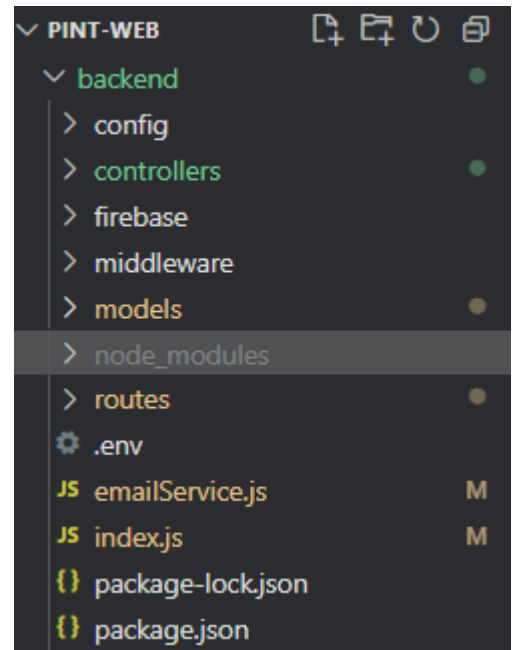


Figura 3-2-Estrutura modular do backend

3.2 Login

A nível de *frontend*, a página de login encontra-se representada na Figura 3-3.

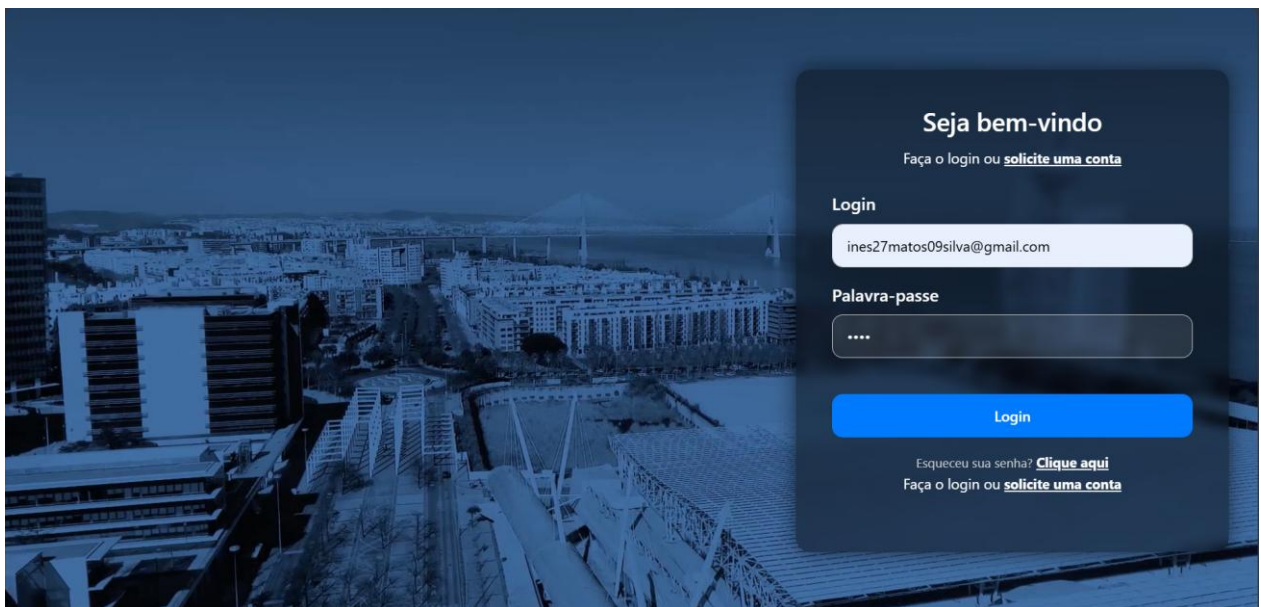


Figura 3-3- Interface do Frontend: Página de Login

A sua implementação pode ser observada na **Figura 3.3**, onde está representado o código responsável pelo processo de autenticação.

Este processo realiza um pedido POST ao endpoint `http://localhost:3000/`, enviando no corpo do pedido o **email** e a **palavra-passe** introduzidos pelo utilizador.

Este endpoint está associado a um *controller* no *backend*, que trata da verificação da existência do utilizador na base de dados. Caso as credenciais estejam corretas, o utilizador é autenticado e redirecionado para a seleção de perfil ou para o processo de primeiro login.

```
9  const Login = () => {
10     const [email, setEmail] = useState("");
11     const [senha, setSenha] = useState("");
12     const [message, setMessage] = useState("");
13     const [messageType, setMessageType] = useState("");
14     const navigate = useNavigate();
15
16     Windsurf: Refactor | Explain | Generate JSDoc | ✕
17     const handleLogin = async (e) => {
18       e.preventDefault();
19       try {
20         const response = await axios.post("http://localhost:3000/", {
21           email: email,
22           senha: senha
23         });
24
25         if (response.data.success) {
26           localStorage.setItem('usuarioId', response.data.user.id);
27           setMessage("Login efetuado com sucesso!");
28           setMessageType("success");
29
30           const primeiroLogin = response.data.user?.primeiroLogin;
31
32           if (primeiroLogin === 0) {
33             navigate("/primeiro-login", { state: { email: email } });
34           } else {
35             console.log("primeiroLogin:", response.data.user?.primeiroLogin);
36
37             const user = response.data.user;
38             navigate("/selecionar-perfil", { state: { user } });
39           }
40         }
41       } else {
42         setMessage("Erro no login: " + response.data.message);
43         setMessageType("error");
44       }
45     } catch (err) {
46       console.error("Erro no login:", err);
47       setMessage("Erro ao efetuar login. Verifique suas credenciais.");
48       setMessageType("error");
49     }
50   }
51 }
```

Figura 3-4- Código da Implementação do Login (Frontend)

No *backend*, foi desenvolvido um *controller* responsável pelo processo de autenticação, conforme representado na Figura 3-5.

Este *controller* inicia por validar se o **email** e a **palavra-passe** foram fornecidos. Caso contrário, é devolvido um erro com código de estado 400.

De seguida, verifica-se a existência do utilizador na base de dados e, em caso positivo, a correspondência da palavra-passe utilizando a biblioteca bcrypt.

Após a autenticação bem-sucedida, o sistema procura todos os perfis associados ao utilizador com base no seu `id_utilizador`. Esta lógica permite preparar o sistema para redirecionar o utilizador para a escolha do perfil, quando aplicável.

Se todas as validações forem concluídas com sucesso, é devolvida uma resposta com código 200 (OK), acompanhada de uma mensagem de sucesso e dos dados relevantes do utilizador.

```
6 const login = async (req, res) => {
7   try {
8     const { email, senha } = req.body;
9
10    if (!email || !senha) {
11      return res.status(400).json({ success: false, message: "Email e palavra-passe são obrigatórios." });
12    }
13    const utilizador = await Utilizador.findOne({ where: { email } });
14    if (!utilizador) {
15      return res.status(404).json({ success: false, message: "Utilizador não encontrado." });
16    }
17    const senhaCorreta = await bcrypt.compare(senha, utilizador.senha);
18
19    if (!senhaCorreta) {
20      return res.status(401).json({ success: false, message: "Palavra-passe incorreta." });
21    }
22    const associacoes = await UtilizadorPerfil.findAll({
23      where: { id_utilizador: utilizador.id_utilizador }
24    });
25    const perfilIds = associacoes.map(assoc => assoc.id_perfil);
26    let perfis = [];
27    if (perfilIds.length > 0) {
28      perfis = await Perfil.findAll({
29        where: { id_perfil: perfilIds }
30      });
31    }
32
33    const perfisFormatados = perfis.map(p => ({
34      id: p.id_perfil,
35      nome: p.nome,
36    }));
37    res.status(200).json({
38      success: true,
39      message: "Login efetuado com sucesso.",
40      user: {
41        id: utilizador.id_utilizador,
42        nome: utilizador.nome,
43        email: utilizador.email,
44        primeiroLogin: utilizador.primeiroLogin,
45        perfis: perfisFormatados,
46      },
47    });
48  } catch (error) {
49    console.error(" Erro no login:", error);
50    res.status(500).json({ success: false, message: "Erro no servidor." });
51  }
52 }
53
54 module.exports = { login };
55
```

Figura 3-5 - Controller Backend do Login

3.3 Primeira Autenticação e Alteração da Palavra-passe

Durante o processo de login, é fundamental verificar se o utilizador está a aceder à aplicação pela primeira vez. Para este fim, foi criada uma coluna adicional na tabela Utilizador, designada por `primeiroLogin`. Esta coluna assume o valor **0** por defeito, indicando que o utilizador ainda não efetuou o login. Após o primeiro acesso, este valor é atualizado para **1**.

Esta lógica permite personalizar a experiência inicial, redirecionando o utilizador para uma página onde é solicitada a alteração da sua palavra-passe. A interface desta funcionalidade encontra-se ilustrada na Figura 3-6.

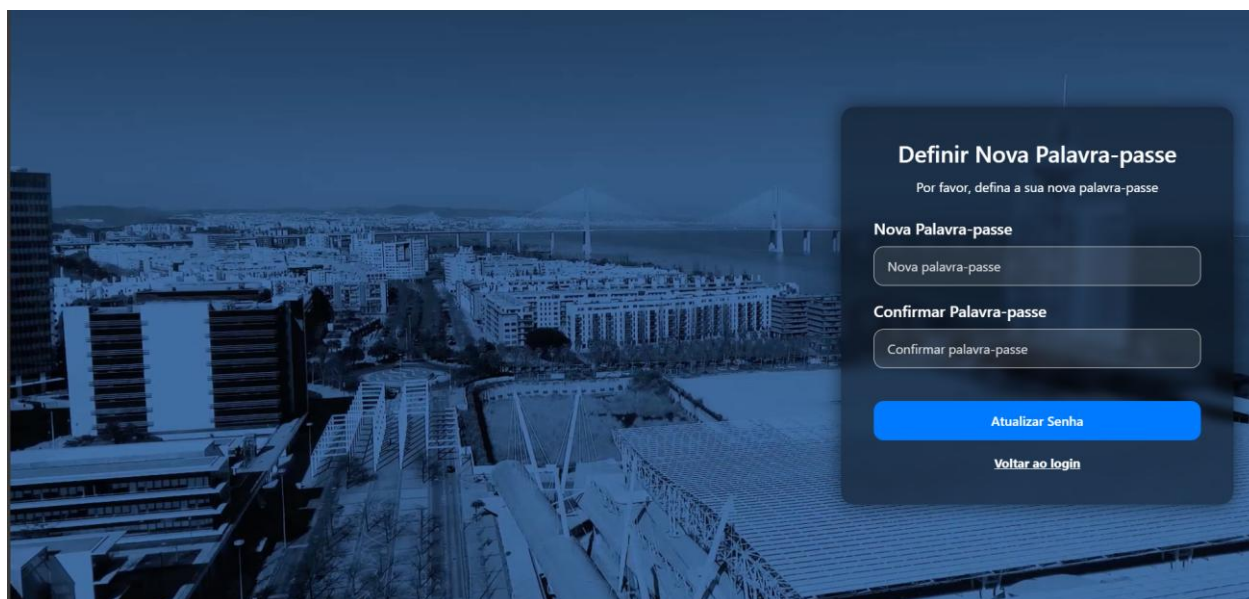


Figura 3-6 - Alteração de Palavra-passe no Primeiro Login

Durante o desenvolvimento da página de alteração de palavra-passe, foi necessário implementar validações no lado do cliente. Como se observa na Figura 3-7, o sistema verifica se os dois campos de palavra-passe introduzidos coincidem. Caso a validação seja bem-sucedida, é efetuado um pedido POST para o endpoint <http://localhost:3000/api/password/first-login>. O corpo do pedido inclui o **email do utilizador** e a **nova palavra-passe**.

```

18     const handleSubmit = async (e) => {
19         e.preventDefault();
20
21         if (newPassword !== confirmPassword) {
22             setMessage("As palavras-passe não coincidem.");
23             setMessageType("error");
24             return;
25         }
26
27         try {
28             const response = await axios.post("http://localhost:3000/api/password/first-login", {
29                 email,
30                 newPassword,
31             });
32
33             if (response.data.success) {
34                 setMessage("Senha atualizada com sucesso. Redirecionando...");
35                 setMessageType("success");
36                 setTimeout(() => navigate("/"), 2000);
37             } else {
38                 setMessage(response.data.message || "Erro ao atualizar senha.");
39                 setMessageType("error");
40             }
41         } catch (err) {
42             console.error("Erro ao atualizar senha:", err);
43             setMessage("Erro ao atualizar senha.");
44             setMessageType("error");
45         }
46     };

```

Figura 3-7- Código: Alteração da Palavra-passe no Primeiro Login (Frontend)

No *backend*, foi desenvolvido um *controller* responsável por processar a alteração da palavra-passe durante o primeiro login, conforme demonstrado na Figura 3-8.

A lógica inicia-se com a pesquisa do utilizador na base de dados através do seu email. Se o utilizador for encontrado, a nova palavra-passe é **encriptada com a biblioteca bcrypt**, garantindo a segurança dos dados. Posteriormente, o campo `primeiroLogin` é atualizado para o valor **1**, assinalando que o primeiro login já foi efetuado com sucesso.

```
const updateFirstLoginPassword = async (req, res) => {
  const { email, newPassword } = req.body;

  try {
    const utilizador = await Utilizador.findOne({ where: { email } });

    if (!utilizador) {
      return res.status(404).json({ success: false, message: "Utilizador não encontrado." });
    }

    const hashedPassword = await bcrypt.hash(newPassword, 12);
    utilizador.senha = hashedPassword;
    utilizador.primeiroLogin = 1;

    await utilizador.save();

    return res.status(200).json({ success: true, message: "Senha atualizada com sucesso." });
  } catch (err) {
    console.error("Erro ao atualizar senha:", err);
    res.status(500).json({ success: false, message: "Erro no servidor." });
  }
};
```

Figura 3-8- Controller Backend: Alteração da Palavra-passe no Primeiro Login

3.4 Escolha do perfil

Após a conclusão do login, o utilizador é redirecionado para uma secção de seleção de perfil, como se pode observar na Figura 3-9.

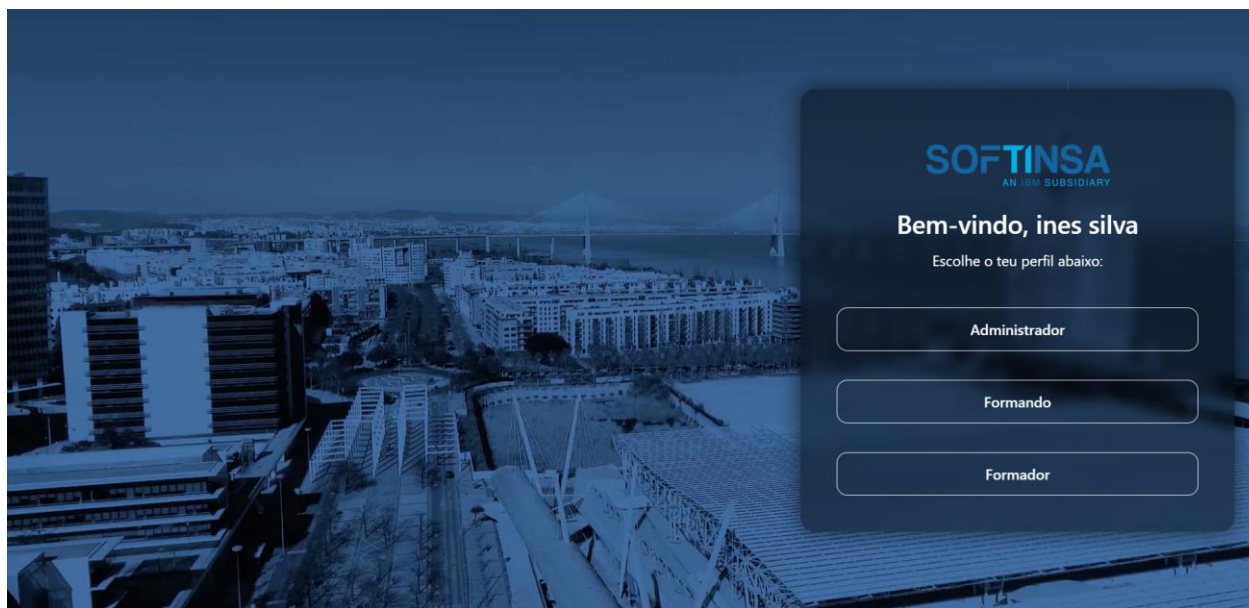


Figura 3-9- Interface de Seleção de Perfil

Esta etapa é essencial, uma vez que os utilizadores podem ter diferentes perfis atribuídos (por exemplo: *Formando*, *Formador* ou *Administrador*), cada um com permissões e funcionalidades específicas dentro da plataforma **The SoftSkills**.

O desenvolvimento desta página tirou partido da estrutura modular já implementada nas páginas de login e alteração de palavra-passe, promovendo a **reutilização de componentes**, estilos e navegação, o que contribuiu para maior eficiência e consistência na interface da aplicação.

No *frontend*, a lógica da seleção de perfil foi implementada no componente `SelecionarPerfil`, que recebe os dados do utilizador autenticado através da navegação com *state*. A interface exibe dinamicamente os perfis disponíveis para o utilizador, permitindo a seleção através de botões estilizados.

Ao selecionar um perfil, o sistema redireciona o utilizador para o *dashboard* correspondente, passando os dados necessários para a nova vista.


```

5  const SelecionarPerfil = () => {
6    const navigate = useNavigate();
7    const location = useLocation();
8    const { user } = location.state || {};
9
10   Windsurf: Refactor | Explain | Generate JSDoc | ✕
11   const handleSelecionar = (perfil) => {
12     console.log("Perfil selecionado:", perfil);
13     // Atualize para usar os paths definidos nas rotas
14     if (perfil.nome.toLowerCase() === 'formando') {
15       navigate('/dashboard/formando', { state: { user, perfil } });
16     } else if (perfil.nome.toLowerCase() === 'administrador') {
17       navigate('/dashboard/administrador', { state: { user, perfil } });
18     } else if (perfil.nome.toLowerCase() === 'formador') {
19       navigate('/dashboard/formador', { state: { user, perfil } });
20     }
21     // Adicione outros perfis conforme necessário
22   };

```

Figura 3-10-Código da interface de seleção de perfil (frontend)

A interface exibe os perfis com base no array `user.perfis`, permitindo uma abordagem dinâmica e escalável. A navegação é realizada através do `useNavigate` da biblioteca `react-router-dom`.

Do lado do *backend*, foi criado um *controller* responsável por listar todos os perfis existentes no sistema, através da consulta à tabela `Perfil`. Esta funcionalidade pode ser útil não apenas para a exibição no momento da escolha do perfil.

```

3  const listarPerfis = async (req, res) => {
4    try {
5      const perfis = await Perfil.findAll({ attributes: ["id_perfil", "nome"], raw: true });
6      res.json(perfis);
7    } catch (error) {
8      console.error("Erro ao listar perfis:", error);
9      res.status(500).json({ erro: "Erro ao buscar perfis." });
10   }
11 };

```

Figura 3-11-Código responsável por devolver os perfis disponíveis

3.5 Pedido de conta e validação do mesmo

Dado que a plataforma **The SoftSkills** é destinada exclusivamente ao uso interno da empresa Softinsa, foi implementado um mecanismo de **pedido de conta** para garantir o controle de acessos.

Este processo permite que potenciais utilizadores solicitem a criação de uma conta introduzindo o seu **email** e o respetivo **número de colaborador**. A submissão do pedido é posteriormente analisada por um **administrador**, que poderá **aprovar** ou **rejeitar** o mesmo. Apenas os administradores têm acesso à gestão dos pedidos pendentes.

Caso o pedido seja aprovado, o utilizador recebe um **email com as credenciais de acesso** e um **link direto para iniciar sessão**. Após o primeiro acesso, é obrigatória a alteração da palavra-passe, reforçando assim a segurança da plataforma.

3.5.1 Pedido de Conta – Interface e Lógica (Frontend)

A interface gráfica para solicitar uma conta encontra-se ilustrada na Figura 3-12, onde o utilizador preenche os dois campos obrigatórios: **email** e **número de colaborador**.

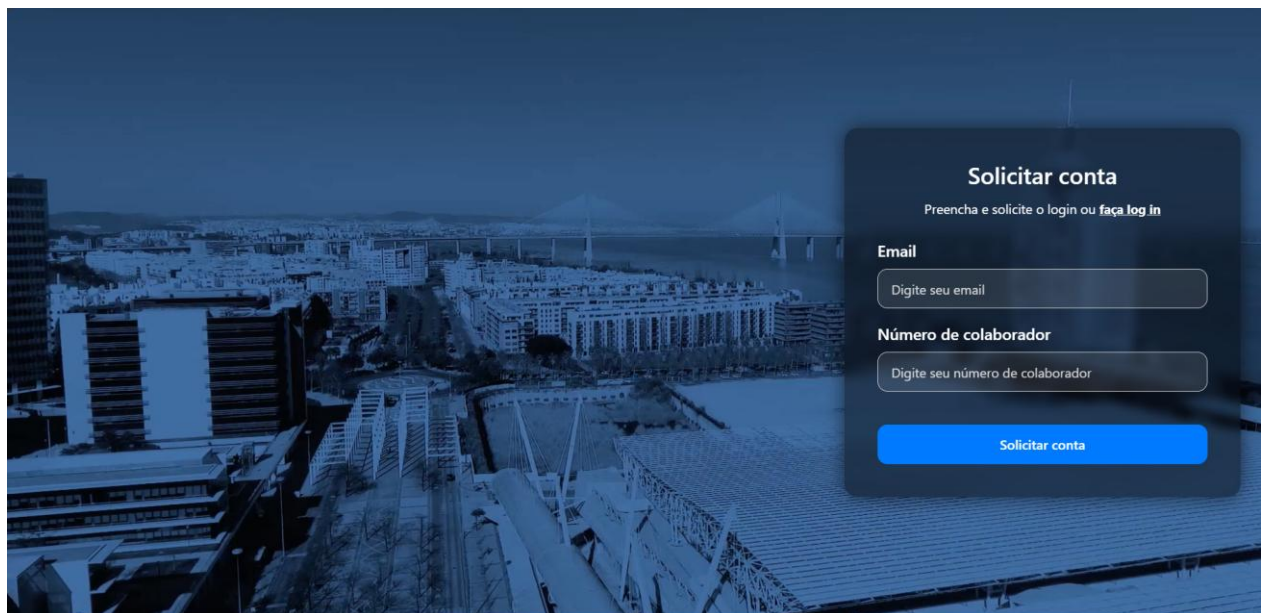


Figura 3-12-Interface de Pedido de Conta (Frontend)

A lógica associada a esta página está implementada no componente SolicitarConta, conforme demonstrado no excerto seguinte:


```

11     const handleSubmit = async (e) => {
12         e.preventDefault();
13
14         try {
15             const resposta = await fetch("http://localhost:3000/api/usuarios/solicitar", {
16                 method: "POST",
17                 headers: {
18                     "Content-Type": "application/json",
19                 },
20                 body: JSON.stringify({ email, numeroColaborador }),
21             });
22
23             const data = await resposta.json();
24
25             if (resposta.ok) {
26                 setMensagem(data.message);
27                 setTipoMensagem("success");
28                 setEmail("");
29                 setNumeroColaborador("");
30             } else {
31                 setMensagem(data.message || "Erro ao solicitar conta.");
32                 setTipoMensagem("danger");
33             }
34         } catch (error) {
35             setMensagem("Erro na requisição.");
36             setTipoMensagem("danger");
37         }
38     };

```

Figura 3-13-Excerto de código do componente SolicitarConta.js

Ao submeter o formulário, é feito um pedido POST para o *endpoint* /api/usuarios/solicitar, com os dados inseridos. O sistema responde com uma mensagem informativa que será apresentada ao utilizador, confirmando o sucesso ou falha da operação.

3.5.2 Registo de Pedido (Backend)

A lógica no backend está a cargo do *controller* solicitarConta, responsável por verificar se já existe um pedido com o mesmo email e, caso contrário, registar o novo pedido na base de dados.

```

147 const solicitarConta = async (req, res) => {
148   try {
149     const { email, numeroColaborador } = req.body;
150
151     const pedidoExistente = await Utilizador.findOne({ where: { email } });
152
153     if (pedidoExistente) {
154       return res.status(400).json({
155         success: false,
156         message: pedidoExistente.pedidoAceitoSN === null ?
157           "Já existe um pedido pendente para este email" :
158           "Este email já possui uma conta ativa"
159       });
160     }
161     await Utilizador.create({
162       email,
163       numeroColaborador,
164       pedidoAceitoSN: null,
165       primeiroLogin: 0
166     });
167
168     return res.status(200).json({
169       success: true,
170       message: "Pedido de conta registrado com sucesso"
171     });
172
173   } catch (error) {
174     console.error("Erro ao processar pedido de conta:", error);
175     return res.status(500).json({
176       success: false,
177       message: "Erro ao processar pedido"
178     });
179   }
180 }
181 };

```

Figura 3-14 -Excerto de código do controller solicitarConta

Neste *controller*, é realizada, em primeiro lugar, uma verificação para confirmar se o email introduzido já se encontra registado na base de dados, através de uma consulta à tabela Utilizador. Caso exista um registo com o mesmo email, o sistema procede à análise do estado da coluna pedidoAceitoSN. Se este campo tiver o valor null, significa que o pedido de conta se encontra pendente de validação por parte do administrador. Por outro lado, se o valor for true ou false, interpreta-se que o email já está associado a uma conta previamente aprovada ou rejeitada, respetivamente.

Caso não exista nenhum registo anterior com o mesmo email, é criado um utilizador na base de dados, com os dados fornecidos no formulário: o email e o numeroColaborador. O campo pedidoAceitoSN é inicializado com o valor null, indicando que o pedido ainda aguarda análise, e o campo primeiroLogin é definido com o valor 0, obrigando o utilizador a alterar a palavra-passe no seu primeiro acesso à aplicação, reforçando assim a segurança do sistema.

3.5.3 Gestão do pedido - Interface e Lógica (Frontend)

A gestão dos pedidos de conta encontra-se integrada na área de administração da plataforma, em conjunto com a gestão geral de utilizadores. Esta funcionalidade é exclusiva para **administradores**, que podem visualizar pedidos pendentes, **aceitar** ou **rejeitar** pedidos, e gerir o estado das contas, assim como editar informações de utilizado e remoção dos mesmos, mas nesta subcapítulo o foco será a gestão dos pedidos.

Uma vez aprovado o pedido, o sistema envia automaticamente um **email com as credenciais temporárias** e um link de acesso à plataforma. Este fluxo garante controlo de acessos e reforça a **segurança e rastreabilidade** do processo de registo.

A interface de gestão está incorporada no componente ListarUtilizadores, responsável por apresentar todos os utilizadores registados, incluindo os que efetuaram pedidos de conta.

Cada utilizador é apresentado com os seus dados principais, perfis atribuídos e estado do pedido (Aceite, Negado ou Pendente). Para pedidos pendentes, o administrador tem acesso a dois botões de ação: **aceitar** ou **rejeitar** o pedido como se observa na Figura 3-15. Nesta figura é possível observar a existência de utilizadores (como, por exemplo, Vera Nunes) que não apresentam qualquer estado associado ao pedido. Tal deve-se ao facto de terem sido criados diretamente pelo administrador, a pedido do próprio, de forma presencial, não tendo passado pelo processo habitual de aceitação ou rejeição de conta.

SOFTINSA - Utilizadores - Cursos - Categorias - Áreas de Formação - Moderar Fórum ines silva	Vera Nunes	vera.nunes@gmail.com	Rua do Povo 18, Coimbra	Formando			
	Paulo Gouveia	paulo.gouveia@gmail.com	Rua da Estação 19, Lisboa	Formando			
	Utilizador teste	testeUser@gmail.com	R. do Palacio do Gelo 3	Administrador, Formando, Formador			
	Ana Silva	ana.silva@gmail.com	Rua das Flores 12, Lisboa	Formando, Formador			
	Joana	Joana@gmail.com	R. do Palacio do Gelo 3	Administrador, Formando			
	Luis	Luis@gmail.com	R. do Palacio do Gelo 3	Formando			
	José Beselga	beselgamain@gmail.com	rua do jose	Formando, Administrador			
	Lola	lola@gmail.com		Sem perfil			
	Inês	pv25222@alunos.estgv.ipv.pt		Formador			
	ines silva	ines27matos09silva@gmail.com	morada e«teste	Administrador, Formando, Formador			
	ines Silva	inessilva2004matos@gmail.com		Administrador, Formando, Formador			
	Pendente	Pedro@gmail.com		Sem perfil			

Figura 3-15-Interface de gestão de pedidos na listagem de utilizadores

Quando o administrador opta por **recusar um pedido de conta**, o sistema executa uma chamada PUT ao *endpoint* correspondente, atualizando o campo `pedidoAceitoSN` na base de dados com o valor 2, que representa o estado **"Negado"**.

Este processo está encapsulado na função `handlePedido`, representada na Figura 3-16, que recebe como parâmetros o id do utilizador e o novo valor do estado do pedido. Caso a atualização seja bem-sucedida (`response.ok`), o estado local da aplicação é atualizado para refletir a mudança imediatamente na interface.

```

226     const handlePedido = async (id, valor) => {
227         try {
228             const response = await fetch(`http://localhost:3000/api/utilizadores/pedido/${id}`, {
229                 method: "PUT",
230                 headers: { "Content-Type": "application/json" },
231                 body: JSON.stringify({ pedidoAceitoSN: valor }),
232             });
233
234             if (response.ok) {
235                 setUtilizadores(prev => prev.map(u =>
236                     u.id_utilizador === id ? { ...u, pedidoAceitoSN: valor } : u
237                 ));
238             } else {
239                 alert("Erro ao atualizar pedido");
240             }
241         } catch (error) {
242             console.error("Erro:", error);
243             alert("Erro ao enviar pedido");
244         }
245     };

```

Figura 3-16- Função para rejeição de pedido

Este mecanismo garante que os pedidos recusados são registrados corretamente, evitando duplicações futuras e proporcionando feedback imediato ao administrador.

Para aprovar um pedido de conta, o administrador deve confirmar a sua identidade através da introdução da **sua própria palavra-passe**. Após esta confirmação, é executada uma chamada POST para o *endpoint* de aceitação formal, que irá tratar da geração da senha temporária, atualização dos dados do utilizador e envio do email com as credenciais.

A função `aceitarPedido` realiza esta operação, enviando o `id_utilizador` e a senha de confirmação no corpo da requisição como podemos observar na Figura 3-17.

```

247     const aceitarPedido = async (idUtilizador, senha) => {
248       try {
249         const response = await fetch("http://localhost:3000/api/utilizadores/admin/aceitar-pedido", {
250           method: "POST",
251           headers: { "Content-Type": "application/json" },
252           body: JSON.stringify({ id_utilizador: idUtilizador, senha })
253         });
254
255         if (response.ok) {
256           alert("Pedido aceito com sucesso!");
257         } else {
258           const errorData = await response.json();
259           alert("Erro ao aceitar pedido: " + (errorData.message || response.statusText));
260         }
261       } catch (error) {
262         console.error("Erro ao aceitar pedido:", error);
263         alert("Erro ao aceitar pedido, verifique o console.");
264       }
265     };

```

Figura 3-17-Função para aceitar pedido de conta

Este fluxo promove a **segurança e a rastreabilidade** do processo de criação de contas, garantindo que apenas administradores autorizados possam aprovar acessos e que todos os utilizadores recebam credenciais personalizadas.

3.5.1 Gestão do pedido (Backend)

A lógica de backend para a gestão dos pedidos é composta por **dois controllers principais**: um para atualizar o estado do pedido (pedidoAceitoSN) e outro para tratar da aceitação completa, incluindo a criação da senha temporária e o envio de email ao utilizador.

Atualização do Estado do Pedido

O primeiro controllers que iremos abordar é o atualizarPedidoAceite que trata da atualização direta do campo pedidoAceitoSN, com base no ID do utilizador. Os estados possíveis são:

- ✓ null: pedido pendente
- ✓ 1: pedido aceite
- ✓ 2: pedido negado

Esta lógica permite ao administrador rejeitar diretamente um pedido ou sinalizar o seu processamento, sem ainda proceder ao envio de credenciais.

```

183 const atualizarPedidoAceite = async (req, res) => {
184   const { id_utilizador } = req.params;
185   const { pedidoAceitoSN } = req.body;
186
187   try {
188     const [linhasAfetadas] = await Utilizador.update(
189       { pedidoAceitoSN },
190       { where: { id_utilizador } }
191     );
192
193     if (linhasAfetadas === 0) {
194       return res.status(404).json({ message: "Utilizador não encontrado." });
195     }
196
197     return res.status(200).json({ message: "Estado do pedido atualizado com sucesso." });
198   } catch (error) {
199     console.error("Erro ao atualizar pedidoAceitoSN:", error);
200     return res.status(500).json({ message: "Erro interno ao atualizar pedido." });
201   }
202 };
203

```

Figura 3-18-Controller para atualização do pedido

Aceitação Completa do Pedido e Envio de Email

A aceitação completa de um pedido de conta é realizada pelo *controller* aceitarPedidoConta, que executa um conjunto de operações encadeadas:

1. Verifica se o pedido ainda não foi processado (pedidoAceitoSN !== 1);
2. Gera uma **senha temporária** aleatória, através do módulo crypto;
3. Encripta essa senha com recurso ao bcrypt, garantindo segurança na base de dados;
4. Atualiza os campos senha, pedidoAceitoSN (com valor 1) e primeiroLogin (com valor 0);
5. Persiste as alterações na base de dados;
6. Envia automaticamente um **email ao utilizador com as credenciais de acesso** e um link direto para a aplicação.

```

234 const aceitarPedidoConta = async (req, res) => {
235   const { id_utilizador } = req.body;
236   console.log("Chamada para aceitarPedidoConta com ID:", id_utilizador);
237
238   try {
239     const utilizador = await Utilizador.findOne({ where: { id_utilizador } });
240
241     if (!utilizador || utilizador.pedidoAceitoSN === true) {
242       return res.status(404).json({ success: false, message: "Pedido não encontrado ou já processado." });
243     }
244
245     const tempPassword = crypto.randomBytes(4).toString("hex");
246     console.log("Senha temporária gerada:", tempPassword);
247
248     const hashedPassword = await bcrypt.hash(tempPassword, 12);
249     utilizador.senha = hashedPassword;
250     utilizador.pedidoAceitoSN = 1;
251     utilizador.primeiroLogin = 0;
252
253     await utilizador.save();
254     console.log("Utilizador salvo com sucesso:", utilizador.id_utilizador);
255
256     await sendAccountAcceptedEmail(utilizador.email, tempPassword).catch(err => {
257       console.error("Erro ao enviar email:", err);
258     });
259
260     return res.status(200).json({ success: true, message: "Conta aprovada e e-mail enviado com sucesso." });
261   } catch (error) {
262     console.error("Erro ao aprovar pedido:", error);
263     return res.status(500).json({ success: false, message: "Erro ao aprovar conta." });
264   }
265 }
266 };

```

Figura 3-19-Controller aceitarPedidoConta

3.5.2 Envio Automático de Email com Credenciais

O envio de credenciais temporárias ao utilizador aquando da aceitação do pedido de conta é realizado automaticamente pelo sistema através da função `sendAccountAcceptedEmail`. Esta funcionalidade garante que os dados de acesso são entregues de forma rápida, segura e personalizada.

Para garantir a segurança das credenciais de envio, as informações sensíveis — nomeadamente o **endereço de email remetente** e a **chave de aplicação** — **não estão codificadas diretamente no projeto**. Em vez disso, são guardadas num ficheiro de variáveis de ambiente. `env`.

```

PINT-WEB > backend > .env
1 EMAIL_USER=carla159silva12222@gmail.com
2 EMAIL_PASS=ykoF plui zssy ayhe

```

Figura 3-20-Exemplo de estrutura no `.env`

A configuração de envio de email é feita com o módulo `nodemailer`, através da criação de um **transportador** (transporter) reutilizável, definido num ficheiro separado. Esta

abordagem modular facilita a manutenção e a reutilização da configuração em diferentes partes do projeto.

```
PINT-WEB > backend > .js emailService.js > ...
1  const nodemailer = require("nodemailer");
2
3  const transporter = nodemailer.createTransport({
4    service: 'gmail',
5    auth: {
6      user: process.env.EMAIL_USER,
7      pass: process.env.EMAIL_PASS,
8    },
9  });
10
11 module.exports = transporter;
```

Figura 3-21-Configuração do transportador de email

Este *transporter* utiliza o serviço do Gmail, autenticando-se com as credenciais presentes nas variáveis de ambiente.

Função de Envio de Email

A função `sendAccountAcceptedEmail` recebe dois parâmetros — o endereço de email do utilizador e a sua senha temporária gerada no momento da aceitação. Esta função é responsável por compor o conteúdo da mensagem e enviá-la utilizando o transporter.

```
204 const sendAccountAcceptedEmail = async (email, password) => {
205   let transporter = nodemailer.createTransport({
206     service: "gmail",
207     auth: {
208       user: process.env.EMAIL_USER,
209       pass: process.env.EMAIL_PASS,
210     },
211   });
212
213   const mailOptions = {
214     from: process.env.EMAIL_USER,
215     to: email,
216     subject: "Conta aprovada - Acesso à plataforma",
217     text: `A sua conta foi aprovada com sucesso!
218
219 Credenciais de acesso:
220 Email: ${email}
221 Senha temporária: ${password}
222
223 Ao entrar pela primeira vez, será solicitado que altere sua senha.
224
225 Acesse: https://frontend-z8p8.onrender.com/
226
227 Obrigado,
228 Equipe de suporte`
229   };
230
231   return transporter.sendMail(mailOptions);
232 };
```

Figura 3-22-Função para envio automático de credenciais por email

Integração com a Lógica de Aceitação de Pedido

A função de envio é utilizada no *controller* aceitarPedidoConta, responsável por concluir o processo de aceitação de um pedido de conta. Esta função:

- Gera uma **palavra-passe temporária aleatória** com crypto;
- Encripta a senha com bcrypt;
- Atualiza o utilizador na base de dados;
- Chama sendAccountAcceptedEmail para enviar o email.

```
234 ✓ const aceitarPedidoConta = async (req, res) => {  
235   const { id_utilizador } = req.body;  
236   console.log("Chamada para aceitarPedidoConta com ID:", id_utilizador);  
237  
238   try {  
239     const utilizador = await Utilizador.findOne({ where: { id_utilizador } });  
240  
241  
242     if (!utilizador || utilizador.pedidoAceitoSN === true) {  
243       return res.status(404).json({ success: false, message: "Pedido não encontrado ou já processado." });  
244     }  
245  
246     const tempPassword = crypto.randomBytes(4).toString("hex");  
247     console.log("Senha temporária gerada:", tempPassword);  
248  
249     const hashedPassword = await bcrypt.hash(tempPassword, 12);  
250     utilizador.senha = hashedPassword;  
251     utilizador.pedidoAceitoSN = 1;  
252     utilizador.primeiroLogin = 0;  
253  
254     await utilizador.save();  
255     console.log("Utilizador salvo com sucesso:", utilizador.id_utilizador);  
256  
257     await sendAccountAcceptedEmail(utilizador.email, tempPassword).catch(err => {  
258       console.error("Erro ao enviar email:", err);  
259     });  
260  
261     return res.status(200).json({ success: true, message: "Conta aprovada e e-mail enviado com sucesso." });  
262   } catch (error) {  
263     console.error("Erro ao aprovar pedido:", error);  
264     return res.status(500).json({ success: false, message: "Erro ao aprovar conta." });  
265   }  
266 };  
267
```

Figura 3-23- Integração do envio de email no processo de aceitação de conta

3.6 SlideBar Administrador

A **Sidebar do Administrador** é um componente central da interface da plataforma **The SoftSkills**, desenvolvido em React. Este componente é responsável pela navegação principal dentro do painel de administração, permitindo o acesso rápido e intuitivo às principais funcionalidades administrativas.

Esta barra lateral foi concebida para garantir uma **experiência de utilizador fluida, responsiva e interativa**, adaptando-se tanto a resoluções amplas como a layouts mais compactos.

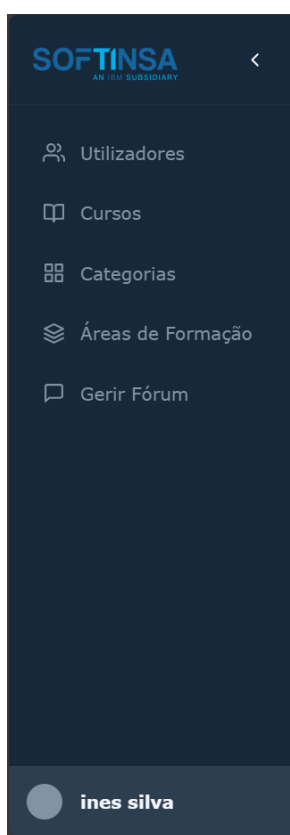


Figura 3-24-Sidebar do Administrador em modo expandido

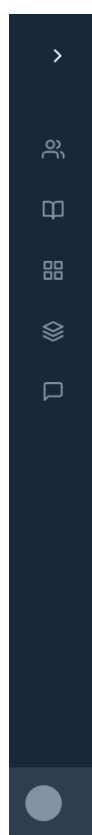


Figura 3-25-Sidebar do Administrador em modo colapsado

A Sidebar é composta por vários elementos dinâmicos que garantem uma interação visual agradável e intuitiva:

- **Colapso e expansão:** A sidebar pode ser recolhida ou expandida através de um botão no topo, utilizando o estado `isCollapsed`. Isto permite maximizar o espaço útil da interface, especialmente em dispositivos com ecrãs mais pequenos.
- **Realce dinâmico do menu:** À medida que o utilizador passa o rato sobre os itens da sidebar, é ativado um efeito visual de destaque (highlight).
- **Navegação por ícones e etiquetas:** Cada item de menu inclui um ícone (através da biblioteca `react-icons`) e uma etiqueta, com rotas definidas para:
 - ✓ Utilizadores
 - ✓ Cursos
 - ✓ Categorias
 - ✓ Áreas de Formação
 - ✓ Gestão de Fórum

A inclusão de ícones nos itens do menu contribui significativamente para uma experiência de navegação mais intuitiva e acessível, permitindo ao utilizador identificar rapidamente as diferentes secções da aplicação, mesmo sem ler o texto. Estes caminhos são definidos no array `menuItems`, e cada ligação (`<Link>`) transporta consigo o estado do user e do perfil, garantindo persistência de contexto entre páginas.

```

42     const menuItems = [
43       { path: "/utilizadores", icon: <FiUsers size={20} />, label: "Utilizadores" },
44       { path: "/cursos", icon: <FiBookOpen size={20} />, label: "Cursos" },
45       { path: "/gerenciar-categorias", icon: <FiGrid size={20} />, label: "Categorias" },
46       { path: "/gerir-areas-formacao", icon: <FiLayers size={20} />, label: "Áreas de Formação" },
47       { path: "/admin/forum", icon: <FiMessageSquare size={20} />, label: "Gerir Fórum" }
48     ];

```

Figura 3-26-Definição dos itens de menu da Sidebar do Administrador com rotas, ícones e etiquetas

A Sidebar é utilizada como **componente de layout envolvente** (`{children}`), permitindo que o conteúdo principal da aplicação seja renderizado à direita da barra lateral. Esta abordagem mantém o menu sempre visível enquanto o utilizador navega entre as páginas administrativas.

Por fim, no rodapé da Sidebar, é apresentada uma secção com o **nome do utilizador que efetuou login**, visível quando a barra está expandida. Esta funcionalidade permite personalizar a interface e reforçar a identificação do utilizador no sistema.

3.7 Slidebar da área Formador e Formando

As restantes Sidebars, associadas aos perfis de **Formador** e **Formando**, mantêm exatamente a estrutura e lógica do componente descrito na secção 3.6 . A única diferença reside nos **itens de menu apresentados**, que são personalizados de acordo com as permissões e necessidades de cada tipo de utilizador. Por exemplo, o menu do Formador inclui apenas as opções relacionadas com a gestão de cursos atribuídos, enquanto o Formando tem acesso ao seu percurso formativo e áreas de avaliação.

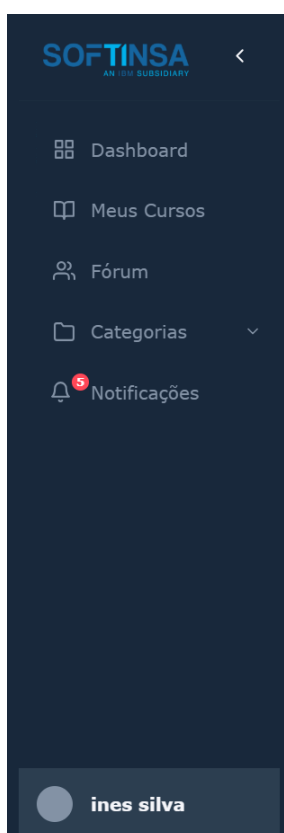


Figura 3-27-Sidebar do Formando

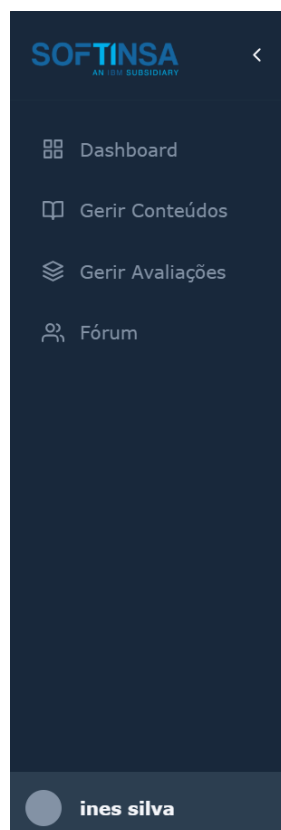


Figura 3-28-Sidebar do Formador

3.8 Gestão de cursos Administrador

Na área do administrador, na seção de cursos, o administrador pode criar cursos, visualizar, editar e excluir cursos existentes, bem como reativar cursos que já foram finalizados.

3.8.1 Criação de novos cursos

Para a criação dos cursos, foi implementado um formulário onde o utilizador insere o título, categoria, área, descrição e formador (no caso de um curso assíncrono, não é necessário seleccionar um formador). Se for seleccionado um formador, é possível adicionar uma breve descrição do mesmo, bem como definir o número de vagas disponíveis no curso. O formulário inclui ainda a definição das datas de início e fim, bem como a possibilidade de efetuar o upload de uma imagem de capa para o curso. Este formulário encontra-se visível na Figura 3-29.

The screenshot displays the 'Criar Novo Curso' (Create New Course) form, a modal window overlaid on the administrator interface. The form contains the following fields and controls:

- Título ***: A text input field for the course title.
- Categoria ***: A dropdown menu with 'Selecione...' as the placeholder.
- Descrição**: A large text area for the course description.
- Área ***: A dropdown menu with 'Selecione...' as the placeholder.
- Formador**: A dropdown menu with 'Nenhum (curso assíncrono)' as the selected option.
- Data de Início ***: A date input field with a calendar icon, showing 'dd/mm/aaaa'.
- Data de Fim ***: A date input field with a calendar icon, showing 'dd/mm/aaaa'.
- Material do Curso (opcional)**: A section with two buttons: 'Escolher Ficheiro' (Choose File) and 'Não foi escolhido nenhum ficheiro' (No file was chosen).
- Buttons**: 'Cancelar' (Cancel) and 'Salvar Curso' (Save Course) buttons at the bottom right.

The background interface shows a sidebar with navigation links: 'Utilizadores', 'Cursos', 'Categorias', 'Áreas de Formação', and 'Gerir Fórum'. The user 'ines silva' is logged in. A table of existing courses is visible in the background, with columns for 'Vagas', 'Tipo', and 'Ações'.

Figura 3-29-Interface do formulário utilizado pelo administrador para a criação de novos cursos

A função `criarCurso` (representada na Figura 3-30) é responsável pela lógica de criação de um novo curso na aplicação. Ela processa os dados recolhidos no formulário, envia-os para a API para serem armazenados no servidor e, caso seja fornecida uma imagem de capa,

chama a função `enviarArquivo` para realizar o upload desse ficheiro associado ao curso.

```
144   const criarCurso = async (e) => {
145     e.preventDefault();
146
147     try {
148       const dadosCurso = {
149         titulo: formData.titulo,
150         descricao: formData.descricao,
151         id_categoria: formData.id_categoria,
152         id_area: formData.id_area,
153         id_formador: formData.id_formador || null,
154         data_inicio: formData.data_inicio,
155         data_fim: formData.data_fim,
156         vagas: formData.vagas,
157         descricao_formador: formData.id_formador ? formData.descricao_formador : null
158       };
159
160       const response = await axios.post("http://localhost:3000/api/cursos/criar", dadosCurso);
161
162       if (formData.ficheiro) {
163         await enviarArquivo(response.data.curso.id_curso, formData.ficheiro);
164       }
165
166       mostrarMensagem("Curso criado com sucesso!", "success");
167       setCursos(prev => [...prev, response.data.curso]);
168       setShowModal(false);
169       resetFormData();
170     } catch (error) {
171       console.error("Erro ao criar curso:", error);
172       const errorMsg = error.response?.data?.message || "Erro ao criar curso";
173       mostrarMensagem(errorMsg, "error");
174     }
175   };

```

Figura 3-30-Função assíncrona que envia os dados do formulário para criar um curso

Nesta função é utilizada uma função auxiliar, `enviarArquivo` (representada Figura 3-31) que recebe o identificador do curso e o ficheiro a enviar, preparando os dados num formato `FormData` para ser submetido à API, que trata o armazenamento do conteúdo relacionado ao curso.

```
243   const enviarArquivo = async (idCurso, arquivo) => {
244     const formDataArquivo = new FormData();
245     formDataArquivo.append("file", arquivo);
246     formDataArquivo.append("id_curso", idCurso);
247     formDataArquivo.append("tipo_conteudo", "material");
248     formDataArquivo.append("descricao", "Material do curso");
249
250     await axios.post("http://localhost:3000/api/conteudo/adicionar", formDataArquivo, {
251       headers: { "Content-Type": "multipart/form-data" }
252     });
253   };

```

Figura 3-31-Função que realiza o envio do conteúdo multimédia relacionado ao curso para a API, garantindo o armazenamento correto do ficheiro

Os controllers apresentados são responsáveis por tratar as requisições recebidas pelas rotas da API, garantindo a criação e gestão dos cursos e conteúdos associados.

O controller representado na Figura 3-32 valida os dados recebidos, distinguindo entre cursos síncronos (com formador e vagas obrigatórias) e assíncronos, cria um registo na base de dados e atualiza o estado do curso. Em caso de sucesso, devolve uma resposta com o novo curso criado; em caso de erro, responde com a respetiva mensagem de falha.

O controller representado na Figura 3-33 trata o upload de múltiplos ficheiros relacionados a um curso, guardando-os num sistema de armazenamento externo e registando as respetivas referências na base de dados. Também responde com a confirmação e os dados dos conteúdos adicionados ou com uma mensagem de erro em caso de falha.


```

26 const criarCurso = async (req, res) => {
27   try {
28     const tipo = req.body.id_formador ? "sincrono" : "assincrono";
29
30     if (tipo === "sincrono" && (!req.body.vagas || req.body.vagas < 1)) {
31       return res.status(400).json({
32         success: false,
33         message: "Para cursos síncronos, é necessário especificar um número válido de vagas (mínimo 1)"
34       });
35     }
36
37     const dadosCurso = {
38       titulo: String(req.body.titulo),
39       descricao: String(req.body.descricao),
40       id_categoria: Number(req.body.id_categoria),
41       id_area: Number(req.body.id_area),
42       id_formador: req.body.id_formador ? Number(req.body.id_formador) : null,
43       descricao_formador: req.body.id_formador ? req.body.descricao_formador : null,
44       data_inicio: new Date(req.body.data_inicio),
45       data_fim: new Date(req.body.data_fim),
46       vagas: tipo === "sincrono" ? Number(req.body.vagas) : null,
47       tipo: tipo,
48       estado: "agendado"
49     };
50
51     const novoCurso = await Curso.create(dadosCurso);
52
53     await atualizarEstadoCurso(novoCurso);
54
55     res.status(201).json({
56       success: true,
57       message: "Curso criado com sucesso!",
58       curso: novoCurso
59     });
60   } catch (error) {
61     console.error("Erro ao criar curso:", error);
62     res.status(500).json({
63       success: false,
64       message: "Erro ao criar curso",
65       error: error.message
66     });
67   }
68 };

```

Figura 3-32-Controller responsável pela validação e criação de um novo curso na base de dados, definindo o tipo de curso e o estado inicial

```

62  const adicionarConteudosMultiplos = async (req, res) => {
63    try {
64      const { id_curso, descricao } = req.body;
65      const files = req.files;
66      if (!files || files.length === 0) {
67        return res.status(400).json({ success: false, message: "Nenhum ficheiro enviado." });
68      }
69      const resultados = [];
70      for (const file of files) {
71        const blob = bucket.file(`conteudos/${uuidv4()}_${file.originalname}`);
72        const blobStream = blob.createWriteStream({
73          metadata: {
74            contentType: file.mimetype,
75          },
76        });
77        await new Promise((resolve, reject) => {
78          blobStream.on("error", reject);
79          blobStream.on("finish", async () => {
80            await blob.makePublic();
81            const publicUrl = `https://storage.googleapis.com/${bucket.name}/${blob.name}`;
82            const conteudo = await ConteudoCurso.create({
83              id_curso,
84              tipo_conteudo: "ficheiro",
85              url: publicUrl,
86              caminho_arquivo: blob.name,
87              descricao: descricao || "Material do curso",
88            });
89
90            resultados.push(conteudo);
91            resolve();
92          });
93          blobStream.end(file.buffer);
94        });
95      }
96      res.status(201).json({
97        success: true,
98        message: "Conteúdos adicionados!",
99        conteudos: resultados
100    });
101    } catch (error) {
102      console.error("Erro ao adicionar múltiplos conteúdos:", error);
103      res.status(500).json({ success: false, message: "Erro no servidor." });
104    }
105  };

```

Figura 3-33-Controller que processa o upload de múltiplos ficheiros para o armazenamento no firebase e regista os conteúdos no sistema, associando-os ao curso correspondente.

4. Plataforma Mobile

5. Conclusão